

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
— o0o —



Báo cáo cá nhân:
UC006 - ReceiveAndReconcileGoods

Hệ thống đặt hàng nhập khẩu

Học phần: Phát triển phần mềm theo chuẩn ITSS

Mã học phần: IT4549

Nhóm 15

Sinh viên thực hiện:

Nguyễn Hữu Nhân - 20235798

Lớp: ITSS

Giảng viên hướng dẫn: TS. Trịnh Tuấn Đạt

Hà Nội - 2026

Mục lục

1	Tài liệu đặc tả yêu cầu phần mềm SRS	3
1.1	Giới thiệu phạm vi nghiệp vụ cá nhân	3
1.2	Phân công nhiệm vụ nhóm	3
1.3	Đặc tả chi tiết Use Case UC006 – ReceiveAndReconcileGoods	3
1.4	Sơ đồ đặc tả	5
2	Tài liệu phân tích Use Case	7
2.1	Mô tả luồng xử lý phân tích	7
2.2	Sơ đồ lớp mức phân tích	7
3	Thiết kế giao diện và Subsystem	9
3.1	Thiết kế và đặc tả màn hình	9
3.1.1	Đặc tả các thành phần giao diện màn hình ReceivingView	10
3.1.2	Giao diện chương trình minh họa	12
3.2	Thiết kế Subsystem	12
4	Thiết kế chi tiết phần mềm	14
4.1	Thiết kế động và tĩnh chi tiết	14
4.2	Biểu đồ phụ thuộc gói	15
5	Nguyên lý thiết kế và mẫu thiết kế áp dụng	17
5.1	Nguyên lý thiết kế	17
5.2	Mẫu thiết kế	17
6	Kiểm thử phần mềm	18
6.1	Thiết kế các ca kiểm thử	18
6.1.1	Thiết kế kiểm thử hộp đen (Black-box Testing)	18
6.1.2	Thiết kế kiểm thử hộp trắng (White-box Testing - Độ bao phủ C1)	19
6.2	Mã nguồn kiểm thử tự động	19

Chương 1

Tài liệu đặc tả yêu cầu phần mềm SRS

1.1 Giới thiệu phạm vi nghiệp vụ cá nhân

Phạm vi công việc cá nhân tập trung vào việc quản lý, đối chiếu số lượng hàng hóa thực tế khi nhập kho so với đơn đặt hàng nước ngoài đã phát hành, ghi nhận các sai lệch thừa thiếu và cập nhật thông tin tồn kho nội bộ của công ty.

1.2 Phân công nhiệm vụ nhóm

Dưới đây là bảng phân công vai trò phụ trách use case của các thành viên trong nhóm:

Bảng 1.1:

Thành viên	MSSV	Use case phụ trách
Nguyễn Hữu Nhân	20235798	UC006 - ReceiveAndReconcileGoods
Vũ Đức Hoàng Anh	20235658	UC001 - ManageImportRequests, UC007 - ManageSystem
Trần Đăng Sinh	20235821	UC003 - CollectInventoryData
Phan Khánh Vũ	20235880	UC004 - PlanImportAllocation
Lương Quốc Khánh	20235754	UC005 - PlaceOverseasOrders
Nguyễn Quý Đức	20235682	UC002 - ManageSiteInformation

1.3 Đặc tả chi tiết Use Case UC006 – ReceiveAndReconcileGoods

Bảng 1.2: Đặc tả Use Case UC-006: ReceiveAndReconcileGoods

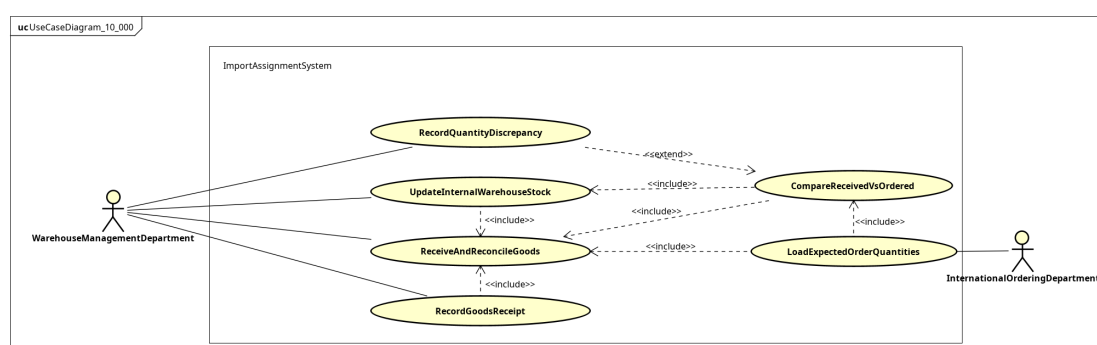
Đặc tả Use Case			
Mã UseCase	UC-006		
Tên UseCase	ReceiveAndReconcileGoods		
Tác nhân	WarehouseManagementDepartment, InternationalOrderingDepartment		
Tiền điều kiện	Đơn đặt hàng đã phát hành cho Site; WarehouseManagementDepartment có quyền tiếp nhận và kiểm hàng.		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	InternationalOrderingDepartment	Cung cấp dữ liệu số lượng đặt kỳ vọng cho WarehouseManagementDepartment.
	2	WarehouseManagementDepartment	Ghi nhận thông tin lô hàng thực tế khi hàng về kho.
	3	System	So sánh số lượng thực nhận với số lượng đã đặt theo từng mặt hàng.
	4	WarehouseManagementDepartment	Xác nhận kết quả kiểm nhận sau khi đối soát.
	5	System	Cập nhật tồn kho nội bộ theo số lượng thực nhận đã xác nhận.
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	3a	System	Phát hiện sai lệch giữa thực nhận và đặt hàng, hệ thống mở bước ghi nhận discrepancy.
	3b	WarehouseManagementDepartment	Ghi chi tiết sai lệch (thiếu/thừa/hư hỏng) và nguyên nhân quan sát được.
Hậu điều kiện	5a	System	Nếu sai lệch vượt ngưỡng, hệ thống gửi cảnh báo cho InternationalOrderingDepartment để xử lý tiếp.
	Kết quả kiểm nhận và đối soát được lưu đầy đủ; tồn kho nội bộ được cập nhật; các sai lệch được ghi nhận và cảnh báo theo quy định.		

* Dữ liệu đầu vào của usecase ReceiveAndReconcileGoods gồm các trường dữ liệu sau:

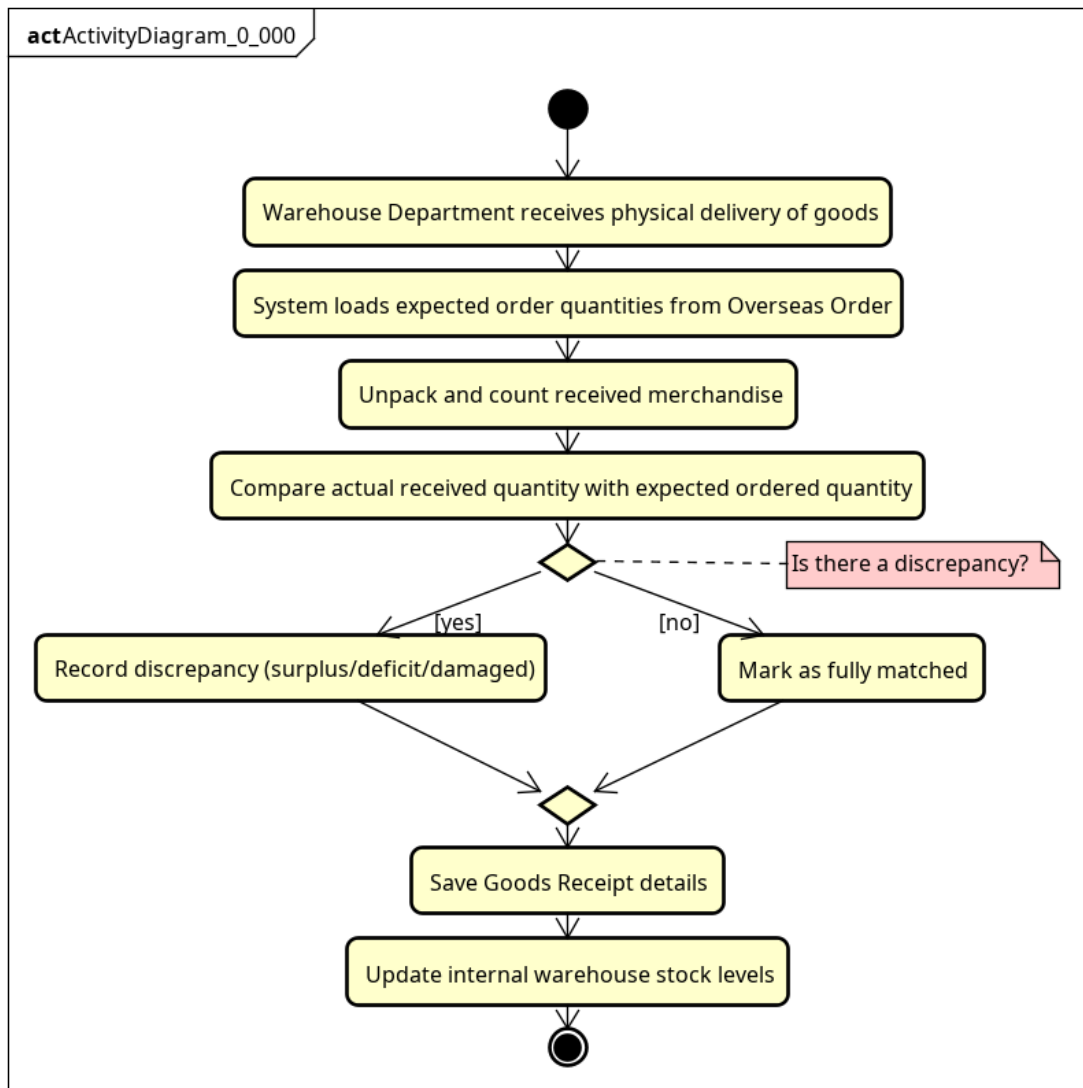
Bảng 1.3: Dữ liệu đầu vào UC-006

STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	orderReference	Mã đơn đặt hàng cần kiểm nhận	Có	Tồn tại và ở trạng thái đã giao/đang giao	PO-2026-0008
2	receivedQuantity	Số lượng thực nhận theo mặt hàng	Có	Số nguyên không âm	145
3	orderedQuantity	Số lượng đã đặt theo đơn	Có	Số nguyên dương	150
4	discrepancyNote	Ghi chú sai lệch nếu có	Không	Bắt buộc khi receivedQuantity khác orderedQuantity	Thiếu 5 đơn vị do hư hỏng vận chuyển

1.4 Sơ đồ đặc tả



Hình 1.1: Use case phân rã UC006 - ReceiveAndReconcileGoods



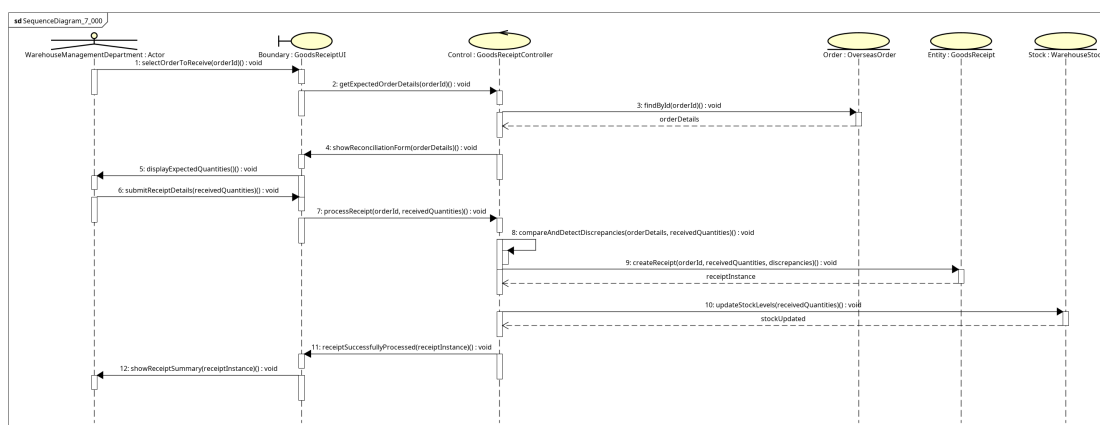
Hình 1.2: Activity diagram UC006 - ReceiveAndReconcileGoods

Chương 2

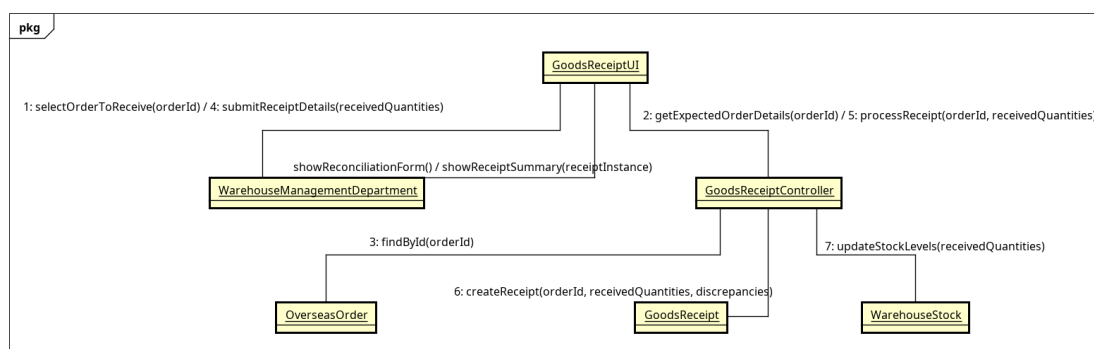
Tài liệu phân tích Use Case

2.1 Mô tả luồng xử lý phân tích

Phân tích luồng xử lý của use case thực hiện bằng sequence diagram và communication diagram để làm rõ các tương tác giữa boundary, control và entity.



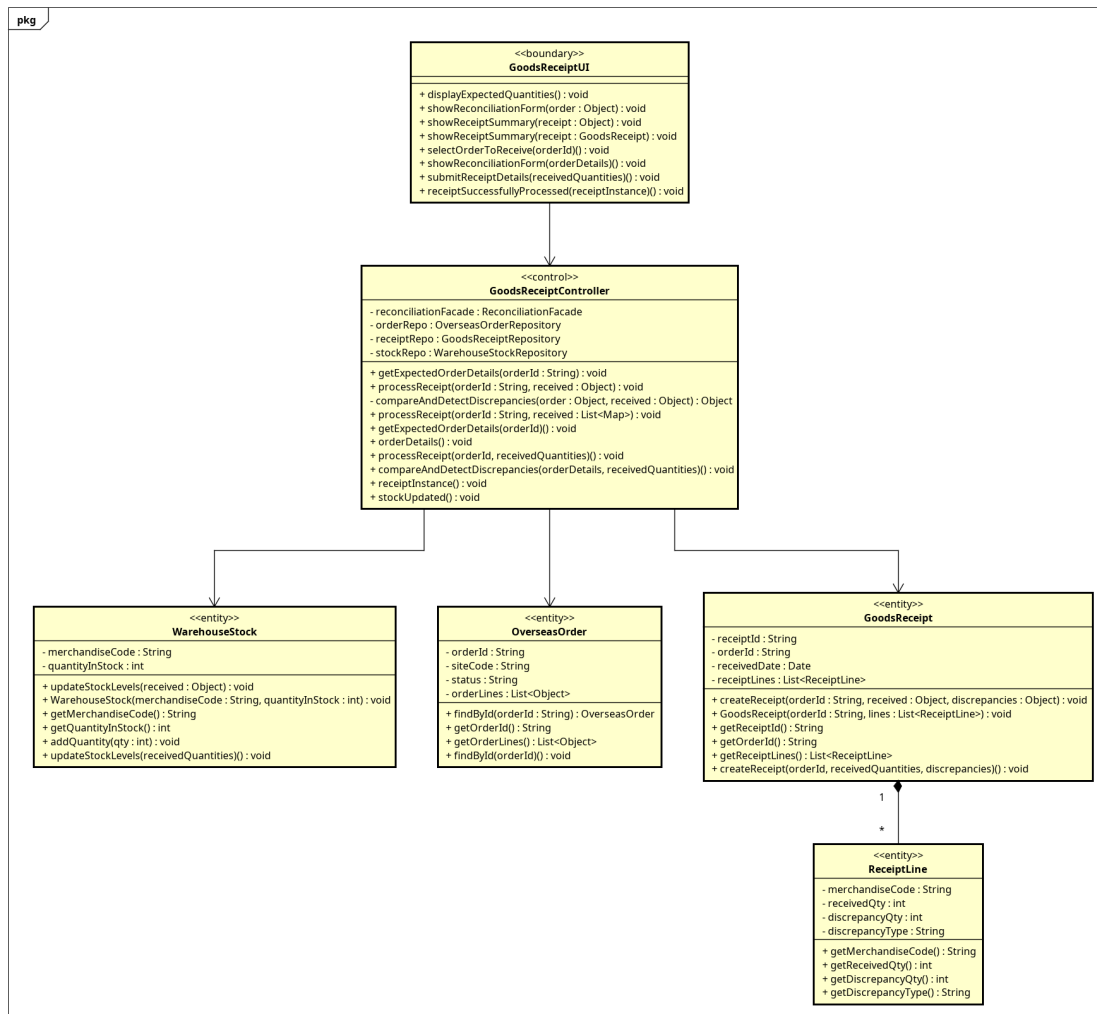
Hình 2.1: Sequence diagram mức phân tích UC006 - ReceiveAndReconcileGoods



Hình 2.2: Communication diagram mức phân tích UC006 - ReceiveAndReconcileGoods

2.2 Sơ đồ lớp mức phân tích

Biểu diễn cấu trúc tĩnh ở mức phân tích nhằm mô tả mối quan hệ giữa các thực thể nghiệp vụ và lớp điều khiển trong use case.



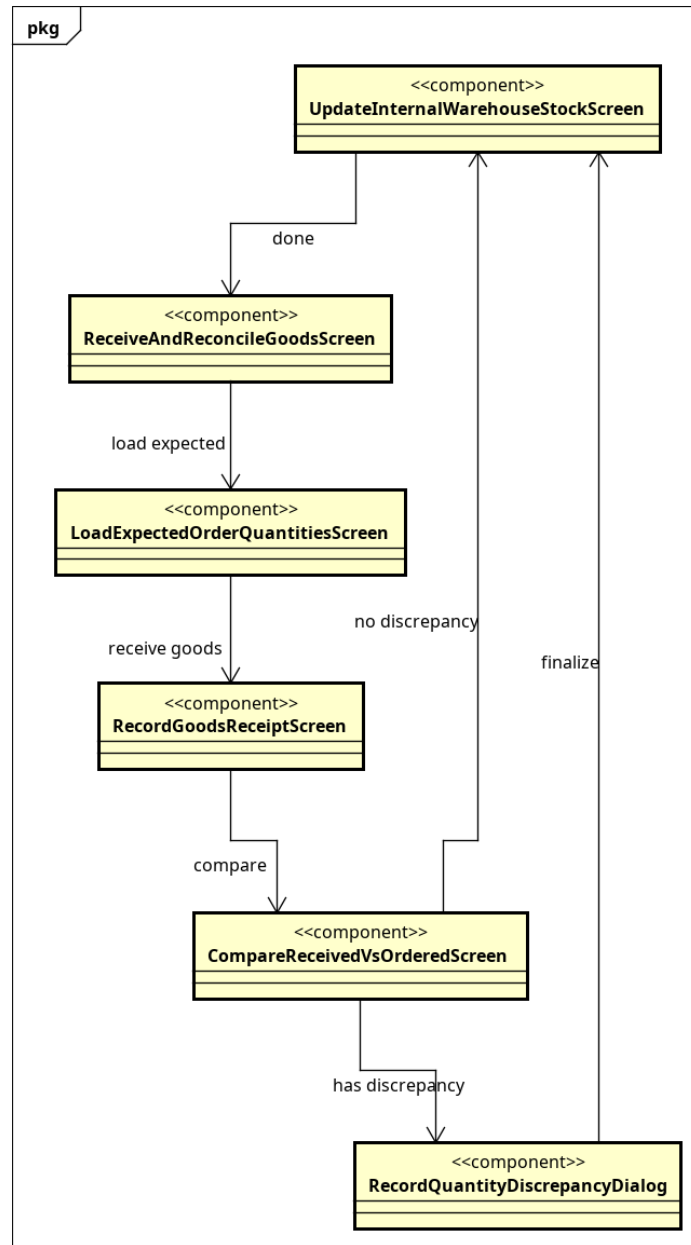
Hình 2.3: Class diagram mức phân tích UC006 - ReceiveAndReconcileGoods

Chương 3

Thiết kế giao diện và Subsystem

3.1 Thiết kế và đặc tả màn hình

Màn hình ReceivingView hỗ trợ việc nhập liệu số lượng thực tế nhận được theo từng mã hàng của đơn hàng.



Hình 3.1: Screen transition diagram UC006 - ReceiveAndReconcileGoods

3.1.1 Đặc tả các thành phần giao diện màn hình ReceivingView

Bảng 3.1:

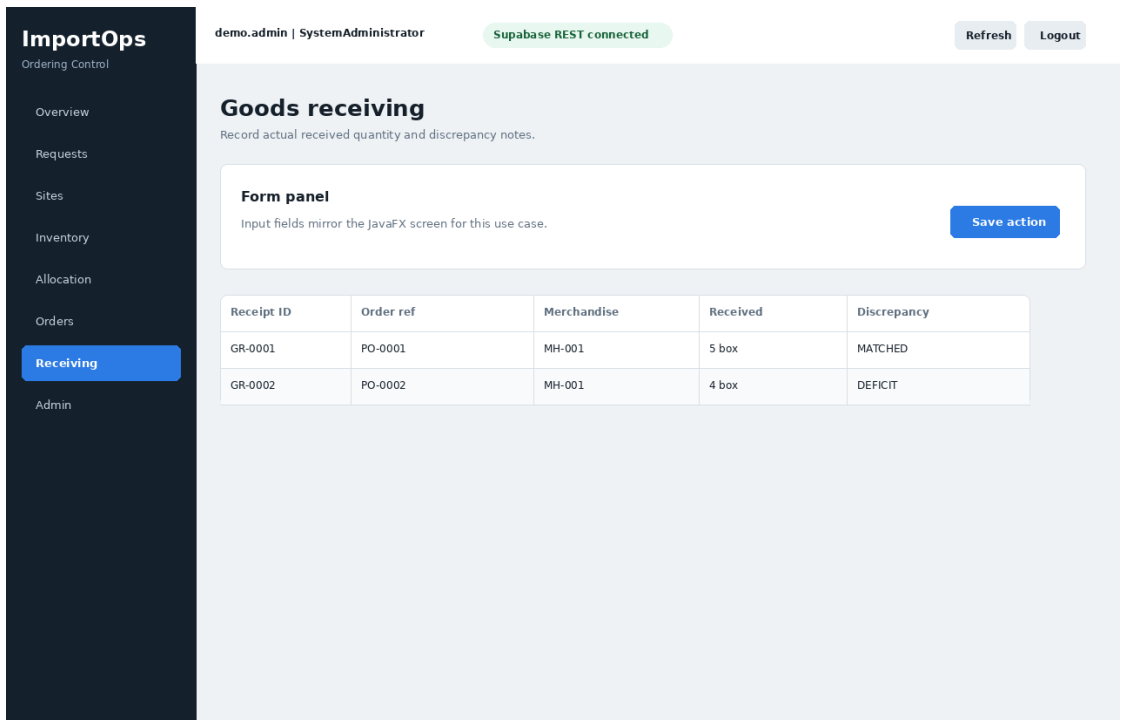
STT	Tên thành phần	Phân loại	Giải thích chi tiết
1	Tiêu đề Receiving	Label	Hiển thị tên màn hình nhận và đối chiếu hàng nhập kho.

(tiếp tục ở trang sau)

Bảng 3.1: (tiếp)

STT	Tên thành phần	Phân loại	Giải thích chi tiết
2	Ô <code>orderReference</code>	Combo Box	Chọn đơn đặt hàng cần kiểm nhận.
3	Bảng <code>ordered lines</code>	Data Grid/Table	Hiển thị số lượng đã đặt theo <code>merchandiseCode</code> .
4	Ô <code>receivedQuantity</code>	Number Field	Nhập số lượng thực nhận, giá trị không âm.
5	Ô <code>discrepancyNote</code>	Text Area	Nhập ghi chú khi số lượng thực nhận khác số lượng đã đặt.
6	Cột <code>discrepancyType</code>	Status Column	Hiển thị Match, Shortage hoặc Surplus theo từng dòng.
7	Nút Reconcile	Button	So sánh <code>receivedQuantity</code> với <code>orderedQuantity</code> .
8	Nút Save receipt	Button	Lưu <code>GoodsReceipt</code> và cập nhật <code>WarehouseStock</code> .
9	Vùng cảnh báo	Message Label	Cảnh báo khi thiếu ghi chú sai lệch hoặc sai lệch vượt ngưỡng.
10	Bảng <code>warehouse stock</code>	Data Grid/Table	Hiển thị tồn kho nội bộ sau khi nhận hàng.

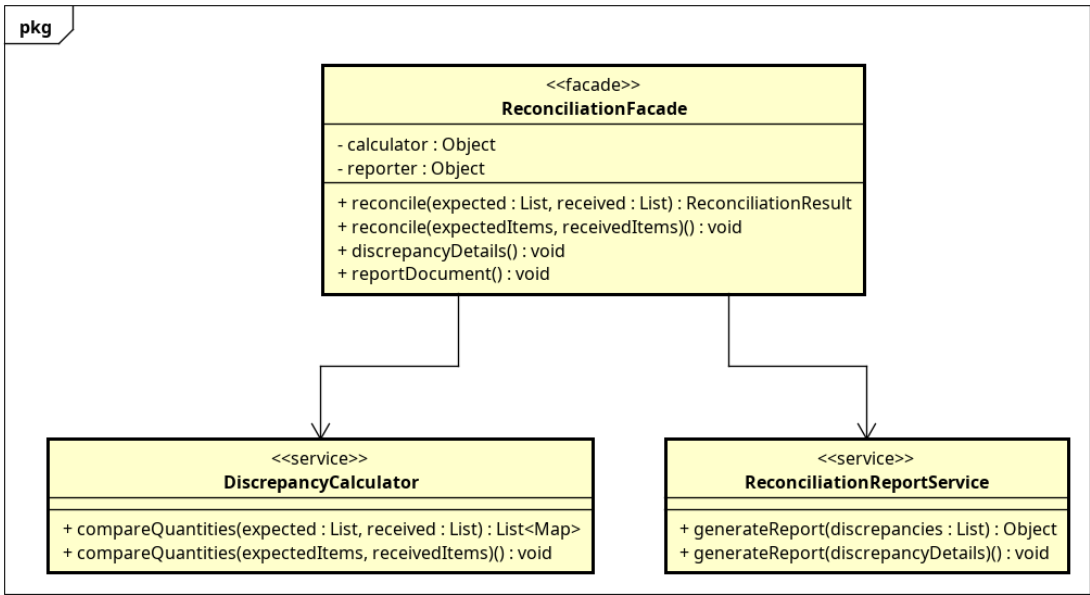
3.1.2 Giao diện chương trình minh họa



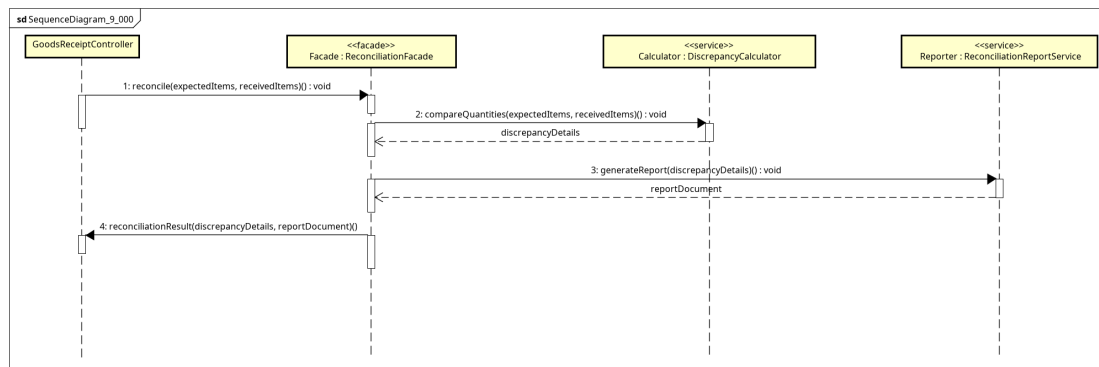
Hình 3.2: Giao diện nhận và đối chiếu hàng nhập kho

3.2 Thiết kế Subsystem

Thiết kế chi tiết cấu trúc tĩnh và động cho subsystem đối chiếu hàng nhập kho phục vụ cho use case.



Hình 3.3: Class diagram subsystem UC006 - ReceiveAndReconcileGoods



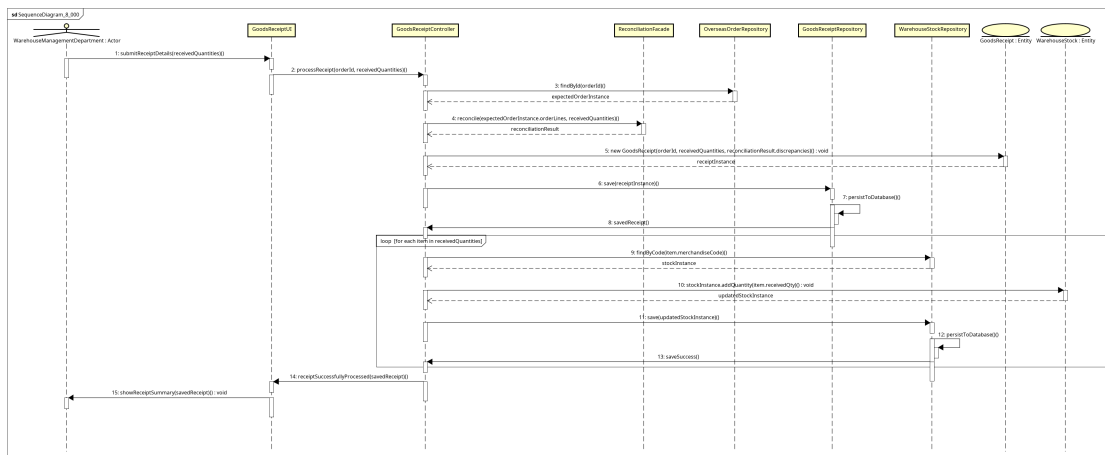
Hình 3.4: Sequence diagram subsystem UC006 - ReceiveAndReconcileGoods

Chương 4

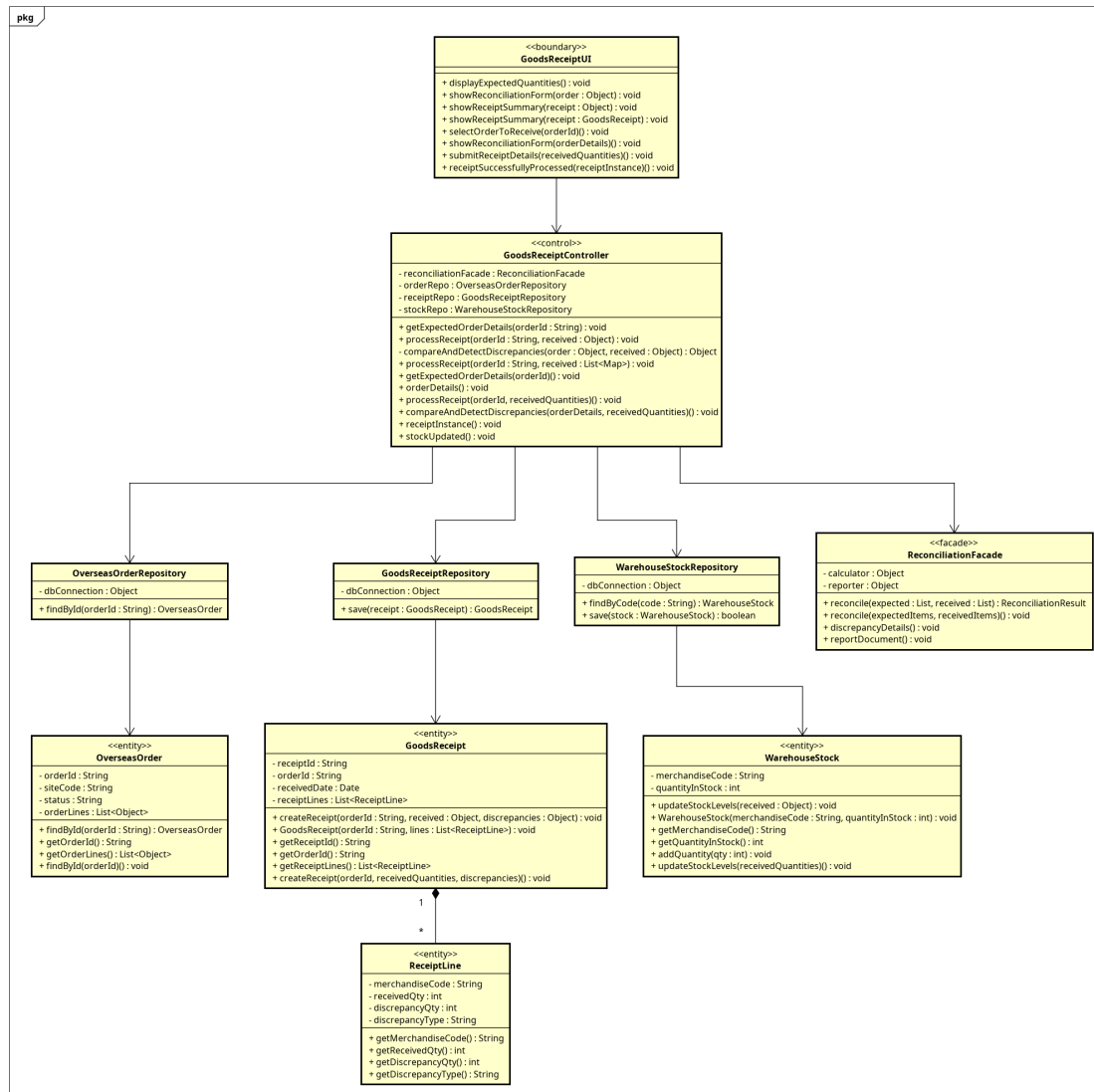
Thiết kế chi tiết phần mềm

4.1 Thiết kế động và tĩnh chi tiết

Chuyển đổi các lớp phân tích sang thiết kế chi tiết ở mức mã nguồn lớp, xác định các thuộc tính, phương thức và mối liên kết.



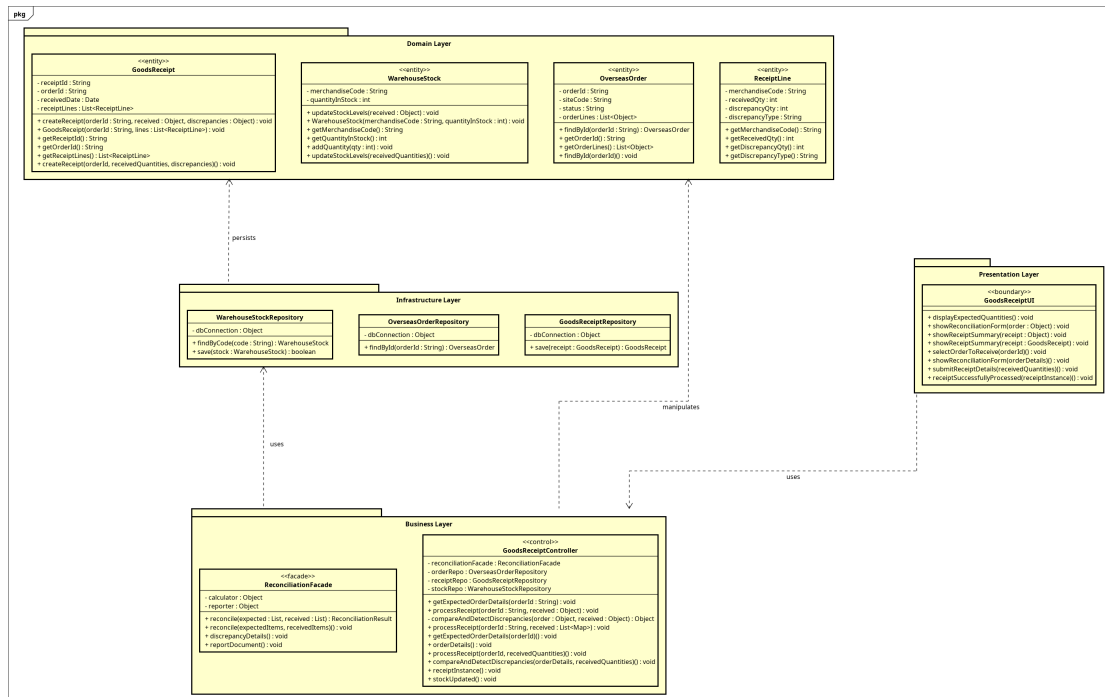
Hình 4.1: Sequence diagram mức thiết kế UC006 - ReceiveAndReconcileGoods



Hình 4.2: Class diagram mức thiết kế UC006 - ReceiveAndReconcileGoods

4.2 Biểu đồ phụ thuộc gói

Sắp xếp các lớp thiết kế vào các gói tương ứng theo cấu trúc kiến trúc đa tầng và chỉ ra mỗi phụ thuộc chiều dọc.



Hình 4.3: Package dependency diagram UC006 - ReceiveAndReconcileGoods

Chương 5

Nguyên lý thiết kế và mẫu thiết kế áp dụng

5.1 Nguyên lý thiết kế

Thiết kế của module UC006 áp dụng triệt để các nguyên lý kiến trúc sạch và lập trình hướng đối tượng:

- **Single Responsibility Principle (SRP):** Tách biệt hoàn toàn logic đối chiếu, kiểm tra sai lệch khỏi tầng hiển thị giao diện. Logic đối so soát nằm trọn vẹn trong các thực thể nghiệp vụ và lớp dịch vụ.
- **Domain Consistency:** Đảm bảo sự nhất quán của mô hình nghiệp vụ. Lớp ‘ReceiptLine’ tự chịu trách nhiệm tính toán sai lệch dựa trên số lượng nhận và số lượng đặt, ngăn ngừa việc ghi nhận thông tin không hợp lệ từ bên ngoài.
- **Dependency Inversion Principle (DIP):** Tầng ứng dụng giao tiếp với tầng lưu trữ dữ liệu thông qua port interface (‘Store’), giúp hệ thống độc lập với hạ tầng cơ sở dữ liệu thật (như Supabase) và dễ dàng thay thế bằng cổng dữ liệu giả lập (‘InMemoryStore’) khi chạy kiểm thử tự động.

5.2 Mẫu thiết kế

Áp dụng mẫu thiết kế để giữ cấu trúc phần mềm rõ ràng và dễ bảo trì:

- **Facade:** Cung cấp một điểm truy cập duy nhất đại diện cho toàn bộ hệ thống dịch vụ qua ‘ImportOrderingFacade’, giúp giảm thiểu mối liên kết giữa tầng giao diện người dùng JavaFX và các lớp dịch vụ nghiệp vụ.
- **Repository (Port and Adapter):** Đóng gói toàn bộ các thao tác truy xuất dữ liệu đằng sau cổng port giao tiếp, hỗ trợ việc chuyển đổi linh hoạt cơ chế lưu trữ.

Chương 6

Kiểm thử phần mềm

6.1 Thiết kế các ca kiểm thử

6.1.1 Thiết kế kiểm thử hộp đen (Black-box Testing)

Thiết kế các ca kiểm thử cho phương thức nhận hàng ‘receive’ trong lớp dịch vụ ‘ReceiptService’:

Bảng 6.1:

Ca kiểm thử	Dữ liệu đầu vào	Kết quả mong đợi	Nhóm phân vùng
TC-UC006-BB1	Số lượng thực nhận bằng số lượng đặt hàng.	Tạo biên nhận thành công, trạng thái khớp (MATCH), không có discrepancy.	Nhận hàng đủ và khớp.
TC-UC006-BB2	Số lượng thực nhận ít hơn đặt hàng, không cung cấp ghi chú sai lệch.	Hệ thống ném lỗi BusinessException bắt buộc nhập ghi chú.	Nhận thiếu hàng không có ghi chú.
TC-UC006-BB3	Số lượng thực nhận ít hơn đặt hàng, có nhập ghi chú đầy đủ.	Tạo biên nhận thành công, loại sai lệch thiếu (SHORTAGE), lưu ghi chú.	Nhận thiếu hàng có ghi chú hợp lệ.
TC-UC006-BB4	Số lượng thực nhận nhiều hơn đặt hàng, không cung cấp ghi chú sai lệch.	Hệ thống ném lỗi BusinessException bắt buộc nhập ghi chú.	Nhận thừa hàng không có ghi chú.
TC-UC006-BB5	Số lượng thực nhận nhiều hơn đặt hàng, có nhập ghi chú đầy đủ.	Tạo biên nhận thành công, loại sai lệch thừa (SURPLUS), lưu ghi chú.	Nhận thừa hàng có ghi chú hợp lệ.

(tiếp tục ở trang sau)

Bảng 6.1: (tiếp)

TC-UC006-BB6	Số lượng thực nhận là số âm (-5).	Hệ thống ném lỗi kiểm tra dữ liệu không hợp lệ.	Giá trị số lượng biên ngoài không hợp lệ.
--------------	-----------------------------------	---	---

6.1.2 Thiết kế kiểm thử hộp trắng (White-box Testing - Độ bao phủ C1)

Thiết kế các ca kiểm thử bao phủ toàn bộ các nhánh logic (Branch Coverage) xử lý sai lệch của phương thức kiểm nhận:

- **Nhánh 1:** Số lượng thực nhận bằng số lượng đặt hàng → Không xảy ra sai lệch, lưu biên nhận bình thường.
- **Nhánh 2:** Số lượng thực nhận khác số lượng đặt hàng và ghi chú sai lệch trống → Ném lỗi 'BusinessException' thông báo bắt buộc nhập ghi chú.
- **Nhánh 3:** Số lượng thực nhận nhỏ hơn số lượng đặt hàng và ghi chú đầy đủ → Xác lập loại sai lệch 'SHORTAGE' và lưu thông tin.
- **Nhánh 4:** Số lượng thực nhận lớn hơn số lượng đặt hàng và ghi chú đầy đủ → Xác lập loại sai lệch 'SURPLUS' và lưu thông tin.

Để bao phủ 4 nhánh này, thiết kế tối thiểu 2 phương thức JUnit test chứa các kịch bản kiểm thử: khớp số lượng và lệch số lượng (thiếu/thừa có/không có ghi chú).

6.2 Mã nguồn kiểm thử tự động

Mã nguồn chương trình kiểm thử tự động bằng JUnit 5 sử dụng cơ chế lưu trữ giả lập bộ nhớ:

```

1 package vn.edu.hust.itss.group15.usecases;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import static org.junit.jupiter.api.Assertions.assertFalse;
5 import static org.junit.jupiter.api.Assertions.assertThrows;
6 import static org.junit.jupiter.api.Assertions.assertTrue;
7
8 import java.time.LocalDate;
9 import java.util.List;
10 import java.util.Map;
11 import org.junit.jupiter.api.Test;
12 import vn.edu.hust.itss.group15.application.BusinessException;
13 import vn.edu.hust.itss.group15.application.
    ImportOrderingFacade;
14 import vn.edu.hust.itss.group15.domain.AllocationPlan;
15 import vn.edu.hust.itss.group15.domain.GoodsReceipt;
16 import vn.edu.hust.itss.group15.domain.ImportRequest;
17 import vn.edu.hust.itss.group15.domain.ImportRequestItem;

```

```

18 import vn.edu.hust.itss.group15.domain.OverseasOrder;
19 import vn.edu.hust.itss.group15.infrastructure.persistence.
    memory.InMemoryStore;
20
21 class Uc006GoodsReceiptTest {
22     @Test
23     void tcUc00601ReceivesMatchedGoodsAndUpdatesWarehouseStock()
24     {
25         ImportOrderingFacade facade = facade();
26         OverseasOrder order = facade.orders().generateOrders(
27             allocationPlanAcrossTwoSites(facade).planId()).getFirst()
28         ;
29         String merchandiseCode = order.orderLines().getFirst().
30             merchandiseCode();
31         int orderedQuantity = order.orderLines().getFirst().
32             quantityOrdered();
33
34         GoodsReceipt receipt = facade.receipts().receive(order.
35             orderId(), Map.of(merchandiseCode, orderedQuantity), Map.
36             of());
37
38         assertFalse(receipt.hasDiscrepancy());
39         assertEquals(orderedQuantity, facade.store().warehouseStock
40             ().getFirst().quantityInStock());
41     }
42
43     @Test
44     void tcUc00602RequiresDiscrepancyNoteForShortOrExcessReceipt
45     () {
46         ImportOrderingFacade facade = facade();
47         OverseasOrder deficitOrder = facade.orders().generateOrders
48             (allocationPlanAcrossTwoSites(facade).planId()).getFirst
49             ();
50         String deficitCode = deficitOrder.orderLines().getFirst().
51             merchandiseCode();
52         int deficitOrdered = deficitOrder.orderLines().getFirst().
53             quantityOrdered();
54
55         assertThrows(BusinessException.class, () -> facade.receipts
56             ().receive(
57                 deficitOrder.orderId(),
58                 Map.of(deficitCode, deficitOrdered - 1),
59                 Map.of()));
60
61         GoodsReceipt surplusReceipt = facade.receipts().receive(
62             deficitOrder.orderId(),
63             Map.of(deficitCode, deficitOrdered + 1),
64             Map.of(deficitCode, "Supplier shipped one extra box"));
65         assertTrue(surplusReceipt.hasDiscrepancy());
66         assertEquals("SURPLUS", surplusReceipt.lines().getFirst().
67             discrepancyType());

```

```

53     }
54
55     private ImportOrderingFacade facade() {
56         return new ImportOrderingFacade(new InMemoryStore());
57     }
58
59     private ImportRequestItem item(String merchandiseCode, int
60         quantity, int daysFromNow) {
61         return new ImportRequestItem(merchandiseCode, quantity, "
62             box", LocalDate.now().plusDays(daysFromNow));
63     }
64
65     private AllocationPlan allocationPlanAcrossTwoSites(
66         ImportOrderingFacade facade) {
67         ImportRequest request = facade.importRequests().create(List
68             .of(item("MH-001", 10, 25)));
69         facade.inventory().recordStock("SITE-SEA-01", "MH-001", 5,
70             "box");
71         facade.inventory().recordStock("SITE-EU-04", "MH-001", 20,
72             "box");
73         return facade.allocation().plan(request.requestId());
74     }
75 }

```